

LẬP TRÌNH WEB (WEBPR330479)

# Kiến trúc 3 tầng và mô hình MVC trong Java



THS. NGUYỄN HỮU TRUNG

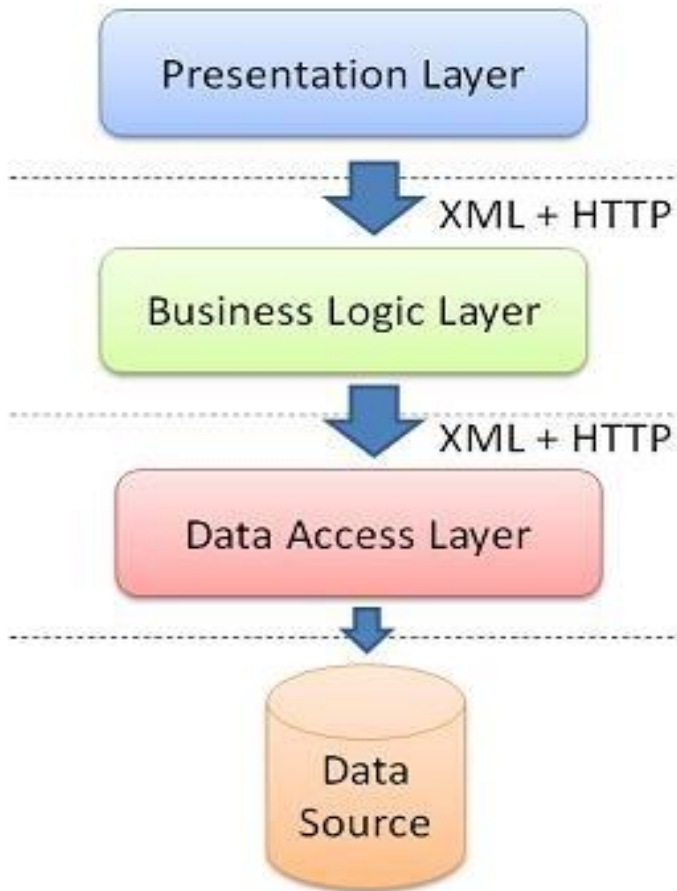
- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- [trungnh@hcmute.edu.vn](mailto:trungnh@hcmute.edu.vn)
- <https://www.youtube.com/@baigiai>



- Kiến trúc 03 tầng (Three Tier Architecture)
- Các mô hình MVC (Model – View – Controller)
- Ví dụ 1: Chức năng đăng nhập tài khoản
- Ví dụ 2: Chức năng đăng ký tài khoản
- Bài tập áp dụng

# Three Tier Architecture

4

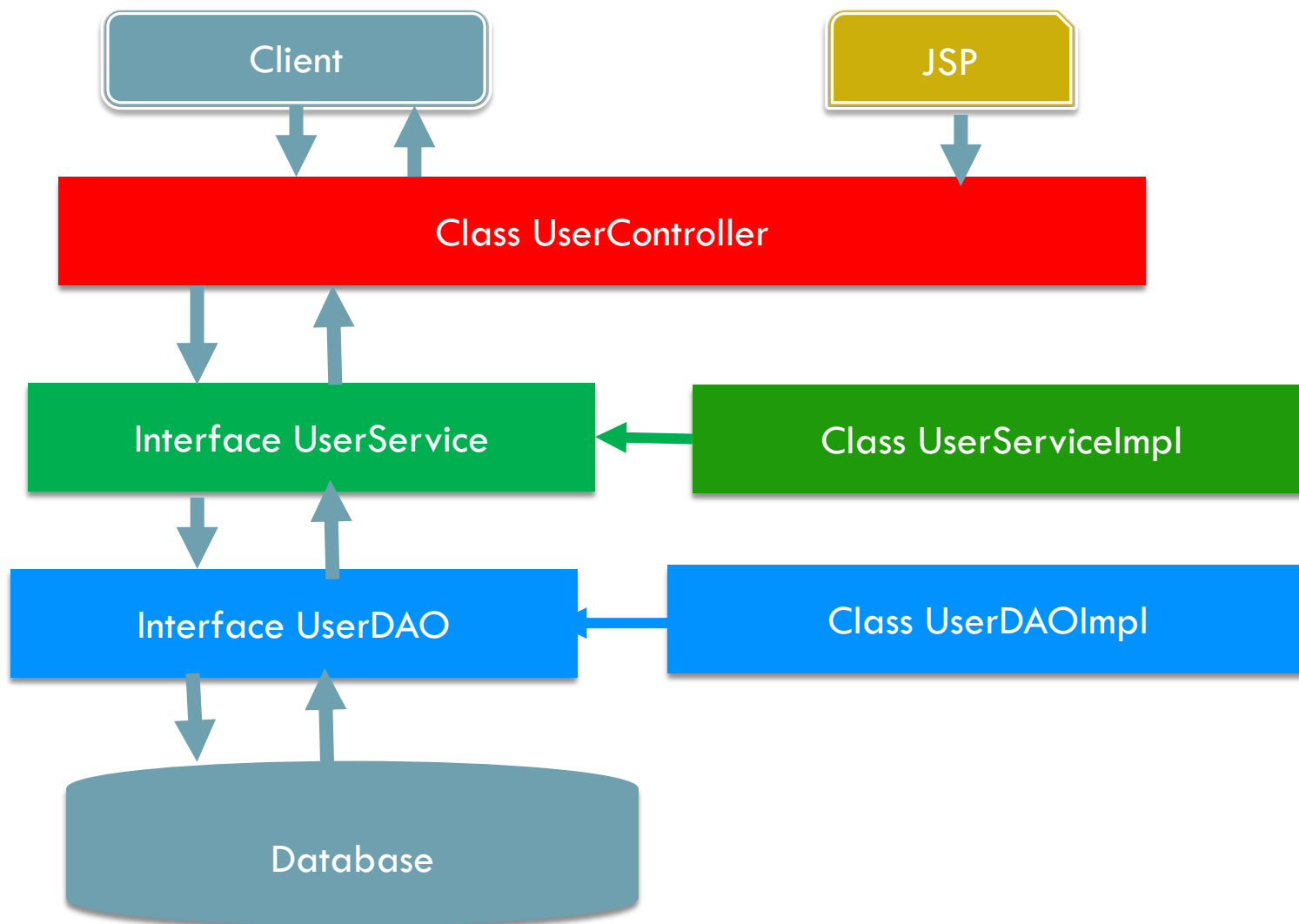


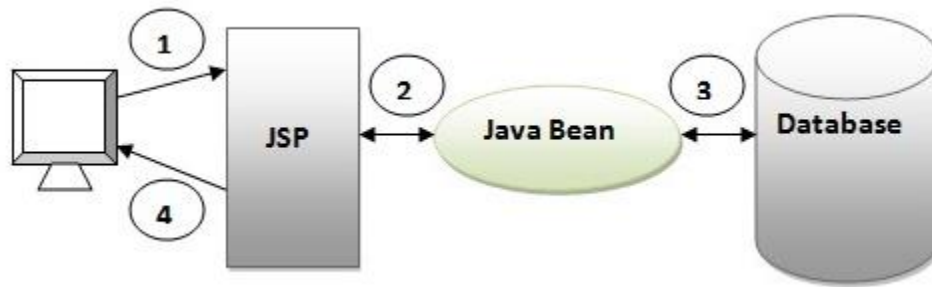
- **Presentation Layer:** lớp giao diện người, lớp này chính là mô hình MVC. Có tác dụng hiển thị view, hiển thị giao diện, lấy data truyền xuống server (controller), kiểm tra thông tin được thì trả về.
- **Business Logic Layer:** xử lý logic. Khi bấm đăng nhập với username và pass (thuộc tầng Presentation) thì data đó sẽ đẩy từ client (view) về server (controller). Khi nó xử lý bằng cách query dữ liệu lên với username và pass đó xem có tồn tại hay không. Nếu có thì nó sẽ GET lên, sau khi GET thì kiểm tra nó thuộc vai trò (Roles) nào. Nếu thuộc admin thì vào trang admin, thuộc web thì vào trang chủ (thuộc lớp Business Logic).
- **Data Access:** Kết nối và query dữ liệu với database rồi chuyển sang cho tầng Business xử lý.

# Three Tier Architecture

5

- Ví dụ: Kiến trúc 03 tầng với chức năng Users





## Nhược điểm:

- **Navigation control is decentralized** bởi vì mỗi trang đều chứa các logic để xác định trang tiếp theo. Nếu tên trang, JSP được thay đổi mà nó được gọi bởi các trang khác, chúng ta cần phải thay đổi nó trong tất cả các trang dẫn đến vấn đề maintenance (bảo trì).

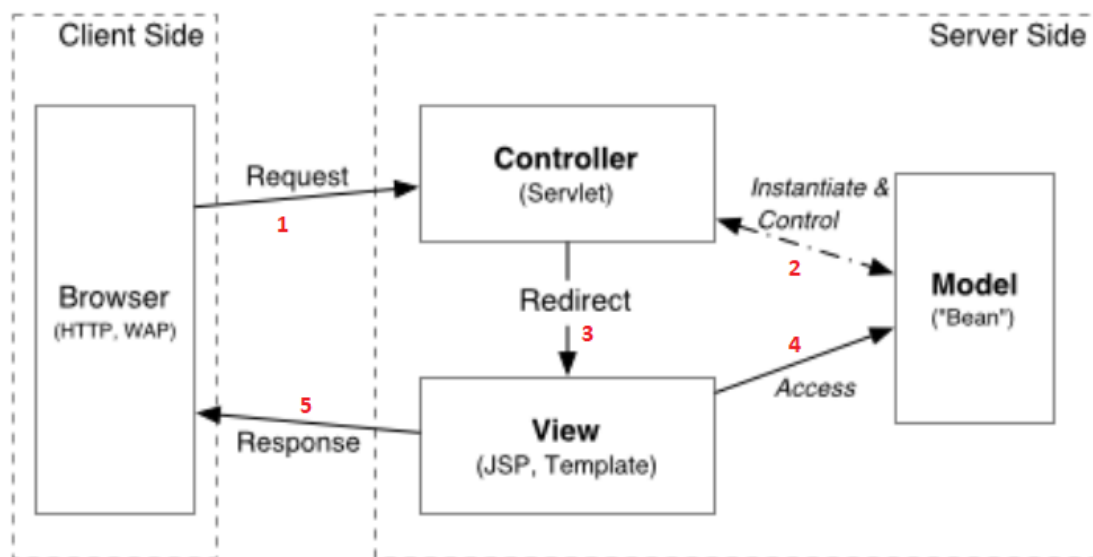
- **Time consuming** Bạn cần phải dành nhiều thời gian để phát triển JSP tags. Vì vậy mà chúng ta không cần phải sử dụng scriptlet tag.

- **Hard to extend** Nó khá ổn cho các ứng dụng nhỏ nhưng cho các ứng dụng lớn sẽ có vấn đề.

- Trình duyệt gửi yêu cầu cho trang JSP.
- JSP truy cập Java Bean và gọi sang business logic.
- Java bean kết nối với cơ sở dữ liệu, nhận và lưu dữ liệu.
- Response được gửi lại cho user bởi JSP.

# Mô hình MVC2

7



- Web Browser send request đến server thông qua các control trên form HTML hay JSP, hay query string url hay qua cookies.
- Servlet – Controller đón nhận request và xác định Model tương ứng để tạo ra instance của JavaBean để đón nhận các giá trị nhập từ request để lưu trữ và xử lý.

- Model thực hiện xử lý, kết nối dữ liệu vật lý dưới DBMS (nếu có) và trả kết quả trả về cho Controller.
- Kết quả xử lý được truyền vào Servlet – Controller, Servlet Controller thực hiện tạo hay lựa chọn View để từ đó đưa kết quả xử lý hay dữ liệu lấy từ Model để cập nhật lại trang kết quả View.
- Controller gửi View qua response cho người dùng để browser có thể trình bày dữ liệu trong Web Browser.

# DEMO

- ▼ ShoppingServletServiceMVC
  - > Deployment Descriptor: ShoppingServletServiceMVC
  - > JAX-WS Web Services
  - ▼ src/main/java
    - > vn.iotstar.connection
    - > vn.iotstar.controller.admin
    - > vn.iotstar.controller.seller
    - > vn.iotstar.controller.web
    - > vn.iotstar.dao
    - > vn.iotstar.dao.impl
    - > vn.iotstar.email
    - > vn.iotstar.filter
    - > vn.iotstar.model
    - > vn.iotstar.service
    - > vn.iotstar.service.impl
    - > vn.iotstar.util
  - > src/main/resources
  - > src/test/java
  - > src/test/resources
  - > JRE System Library [JavaSE-1.8]
  - > Maven Dependencies
  - > Server Runtime [Apache Tomcat v9.0]
  - > Deployed Resources
  - > src
  - > target
    - pom.xml

- ▼ webapp
  - > ckeditor
  - ▼ common
    - > admin
    - > web
      - taglib.jsp
  - ▼ decorators
    - admin.jsp
    - login.jsp
    - web.jsp
  - ▼ template
    - > admin
    - ▼ web
      - > static
  - ▼ views
    - > admin
    - ▼ web
      - cart.jsp
      - home.jsp
      - home1.jsp
      - index.jsp
      - list-cart.jsp
      - myaccount.jsp
      - orderbuy-list.jsp
      - product-detail.jsp
      - product-grid.jsp
      - product-list.jsp
      - productSeachByCategory.jsp
      - product-seach-by-name.jsp
      - login.jsp
      - login2.jsp
      - register.jsp
  - ▼ WEB-INF
    - decorators.xml
    - web.xml
    - index.jsp

# Ví dụ: Chức năng đăng nhập tài khoản

9

## □ Bước 1: Thiết kế view: login.jsp

Đăng Nhập Vào Hệ Thống



trungnh



.....

☐ Nhớ tôi

[Quên mật khẩu?](#)

Đăng nhập

Nếu bạn chưa có tài khoản trên hệ thống, thì hãy [Đăng ký](#)

```
<form action="login" method="post">
<h2>Tạo tài khoản mới</h2>
<c:if test="${alert !=null}">
    <h3 class="alert alert-  
danger">${alert}</h3>
</c:if>
<section>
<label class="input login-input">
<div class="input-group">
<span class="input-group-addon"><i class="fa  
fa-user"></i></span>
<input type="text" placeholder="Tài khoản"
name="username"
class="form-control">
</div>
</label>
</section>
```

## □ Bước 2: Tạo Model: User.java

```
@SuppressWarnings("serial")
public class User implements Serializable {
    private int id;
    private String email;
    private String userName;
    private String fullName;
    private String passWord;
    private String avatar;
    private int roleid;
    private String phone;
    private Date createDate;

    //Tạo constructor, getters/setters
```

## □ Bước 3: Định nghĩa hàm trong interface service

```
public interface UserService {  
  
    User login(String username, String password);  
    User get(String username);  
}
```

## □ Bước 4: Viết hàm thực thi trong service implement

```
public class UserServiceImpl implements UserService {  
    UserDao userDao = new UserDaoImpl();  
  
    //viết các hàm đã định nghĩa bên interface service tại đây  
}
```

## □ Bước 4: Viết hàm thực thi trong service implement

```
@Override
public User login(String username, String password) {
    User user = this.get(username);
    if (user != null && password.equals(user.getPassword())) {
        return user;
    }

    return null;
}
```

```
@Override
public User get(String username) {
    return userDao.get(username);
}
```

## □ Bước 5: Định nghĩa hàm trong interface DAO

```
public interface UserDao {  
    User get(String username);  
}
```

## □ Bước 6: Viết hàm thực thi trong DAO Implement

```
public class UserDaoImpl implements UserDao {  
    public Connection conn = null;  
    public PreparedStatement ps = null;  
    public ResultSet rs = null;  
  
    //Viết hàm xử lý DAO tại đây  
}
```

## □ Bước 6: Viết hàm thực thi trong DAO Implement

@Override

```
public User get(String username) {  
    String sql = "SELECT * FROM [User] WHERE username = ? ";  
    try {  
        conn = new DBConnection().getConnection();  
        ps = conn.prepareStatement(sql);  
        ps.setString(1, username);  
        rs = ps.executeQuery();  
        while (rs.next()) {  
            User user = new User();  
            user.setId(rs.getInt("id"));  
            user.setEmail(rs.getString("email"));  
            user.setUserName(rs.getString("username"));  
            user.setFullName(rs.getString("fullname"));  
            user.setPassword(rs.getString("password"));  
            user.setAvatar(rs.getString("avatar"));  
            user.setRoleId(Integer.parseInt(rs.getString("roleid")));  
            user.setPhone(rs.getString("phone"));  
            user.setCreateDate(rs.getDate("createdDate"));  
            return user;        }  
    } catch (Exception e) {e.printStackTrace();        }  
    return null;  
}
```

## □ Bước 7: Viết Controller : LoginController.java

```
@SuppressWarnings("serial")
@WebServlet(urlPatterns = "/login")
public class LoginController extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
    ServletException, IOException {

        HttpSession session = req.getSession(false);
        if (session != null && session.getAttribute("account") != null) {
            resp.sendRedirect(req.getContextPath()+ "/waiting");
        }
        return;
    }
}
```

```
// Check cookie
Cookie[] cookies = req.getCookies();
if (cookies != null) {
    for (Cookie cookie : cookies) {
        if (cookie.getName().equals("username")) {
            session = req.getSession(true);
            session.setAttribute("username", cookie.getValue());
            resp.sendRedirect(req.getContextPath()+ "/waiting");
        }
    }
    return;
}
req.getRequestDispatcher("views/login.jsp").forward(req, resp);
}
```

## □ Bước 7: Viết Controller : LoginController.java

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse
resp) throws ServletException, IOException {
    resp.setContentType("text/html");
    resp.setCharacterEncoding("UTF-8");
    req.setCharacterEncoding("UTF-8");
    String username = req.getParameter("username");
    String password = req.getParameter("password");
    boolean isRememberMe = false;
    String remember = req.getParameter("remember");

    if("on".equals(remember)){
        isRememberMe = true;
    }
    String alertMsg="";
```

# Ví dụ: Chức năng đăng nhập tài khoản

17

## □ Bước 7: Viết Controller : LoginController.java

```
if(username.isEmpty() || password.isEmpty()){
    alertMsg = "Tài khoản hoặc mật khẩu không được rỗng";
    req.setAttribute("alert", alertMsg);
    req.getRequestDispatcher("/views/login.jsp").forward(req, resp);
    return;
}
UserService service = new UserServiceImpl();
User user = service.login(username, password);
if(user!=null){
    HttpSession session = req.getSession(true);
    session.setAttribute("account", user);
    if(isRememberMe){
        saveRemeberMe(resp, username);
    }
    resp.sendRedirect(req.getContextPath()+"/waiting");
}else{
    alertMsg = "Tài khoản hoặc mật khẩu không đúng";
    req.setAttribute("alert", alertMsg);
    req.getRequestDispatcher("/views/login.jsp").forward(req, resp);
}
}
```

## □ Bước 7: Viết Controller : LoginController.java

```
private void saveRememberMe(HttpServletRequest response, String
username){
    Cookie cookie = new Cookie(Constant.COOKIE_REMEMBER,
username);
    cookie.setMaxAge(30*60);
    response.addCookie(cookie);
}
```

```
public static final String SESSION_USERNAME = "username";
public static final String COOKIE_REMEMBER = "username";
```

## □ Bước 9: Viết Controller : WaitingController.java

```
@SuppressWarnings("serial")
@WebServlet(urlPatterns="/waiting")
public class WaitingController extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
    ServletException, IOException {
        HttpSession session= req.getSession();
        if(session != null && session.getAttribute("account") != null) {
            User u=(User) session.getAttribute("account");
            req.setAttribute("username", u.getUserName());
            if(u.getRoleid()==1) {
                resp.sendRedirect(req.getContextPath()+"/admin/home");
            }else if(u.getRoleid()==2) {
                resp.sendRedirect(req.getContextPath()+"/manager/home");
            }else {
                resp.sendRedirect(req.getContextPath()+"/home");
            }
        }else {
            resp.sendRedirect(req.getContextPath()+"/login");
        }
    }
}
```

## □ Bước 10: Điều chỉnh và gọi Session trên trang topbar.jsp

```
<c:choose>
<c:when test="{sessionScope.account == null}">
  <div class="col-sm-6">
    <ul class="list-inline right-topbar pull-right">
      <li><a href="{pageContext.request.contextPath }/login">Đăng nhập</a>
      | <a href="{pageContext.request.contextPath }/register">Đăng ký</a></li>
      <li><i class="search fa fa-search search-button"></i></li>
    </ul>
  </div>
</c:when>
<c:otherwise>
  <div class="col-sm-6">
    <ul class="list-inline right-topbar pull-right">
      <li><a href="{pageContext.request.contextPath
}/member/myaccount">${sessionScope.account.fullName}</a> | <a
href="{pageContext.request.contextPath }/logout">Đăng Xuất</a></li>
      <li><i class="search fa fa-search search-button"></i></li>
    </ul>
  </div>
</c:otherwise></c:choose>
```

# Ví dụ 2: Chức năng đăng ký tài khoản

21

## □ Bước 1: Thiết kế view: register.jsp

Tạo tài khoản mới

|                                                                                   |                                                    |
|-----------------------------------------------------------------------------------|----------------------------------------------------|
|  | <input type="text" value="trungnh"/>               |
|  | <input type="text" value="Họ tên"/>                |
|  | <input type="text" value="Nhập Email"/>            |
|  | <input type="text" value="....."/>                 |
|  | <input type="password" value="Mật khẩu"/>          |
|  | <input type="password" value="Nhập lại mật khẩu"/> |
| <input type="button" value="Tạo tài khoản"/>                                      |                                                    |

Nếu bạn đã có tài khoản? [Đăng nhập](#)

```
<form action="register" method="post">
<h2>Tạo tài khoản mới</h2>
<c:if test="${alert !=null}">
    <h3 class="alert alert-
        danger">${alert}</h3>
</c:if>
<section>
<label class="input login-input">
<div class="input-group">
<span class="input-group-addon"><i
class="fa fa-user"></i></span>
<input type="text" placeholder="Tài
khoản" name="username"
class="form-control">
</div>
</label>
</section>
```

## □ Bước 2: Tạo Model: User.java

```
@SuppressWarnings("serial")
public class User implements Serializable {
    private int id;
    private String email;
    private String userName;
    private String fullName;
    private String passWord;
    private String avatar;
    private int roleid;
    private String phone;
    private Date createdAt;

    //Tạo constructor, getters/setters
}
```

## Ví dụ 2: Chức năng đăng ký tài khoản

23

### □ Bước 3: Định nghĩa hàm trong interface service

```
public interface UserService {  
    void insert(User user);  
    boolean register(String email, String password, String username, String  
        fullname, String phone);  
    boolean checkExistEmail(String email);  
    boolean checkExistUsername(String username);  
    boolean checkExistPhone(String phone);  
}
```

### □ Bước 4: Viết hàm thực thi trong service implement

```
public class UserServiceImpl implements UserService {  
    UserDao userDao = new UserDaoImpl();  
  
    //viết các hàm đã định nghĩa bên interface service tại đây  
  
}
```

### □ Bước 4: Viết hàm thực thi trong service implement

```
@Override
public boolean register(String username, String password, String email, String
fullname, String phone ) {
    if (userDao.checkExistUsername(username)) {
        return false;
    }
    long millis=System.currentTimeMillis();
    java.sql.Date date=new java.sql.Date(millis);
    userDao.insert(new User(email, username, fullname,password,
    null,5,phone,date));
    return true;
}
```

## □ Bước 4: Viết hàm thực thi trong service implement

```
public boolean checkExistEmail(String email) {  
    return userDao.checkExistEmail(email);  
}  
public boolean checkExistUsername(String username) {  
    return userDao.checkExistUsername(username);  
}  
@Override  
public boolean checkExistPhone(String phone) {  
    return userDao.checkExistPhone(phone);  
}  
@Override  
public void insert(User user) {  
    userDao.insert(user);  
}
```

### □ Bước 5: Định nghĩa hàm trong interface DAO

```
public interface UserDao {  
    void insert(User user);  
    boolean checkExistEmail(String email);  
    boolean checkExistUsername(String username);  
    boolean checkExistPhone(String phone);  
}
```

### □ Bước 6: Viết hàm thực thi trong DAO Implement

```
public class UserDaoImpl implements UserDao {  
    public Connection conn = null;  
    public PreparedStatement ps = null;  
    public ResultSet rs = null;  
  
    //Viết hàm xử lý DAO tại đây  
}
```

## □ Bước 6: Viết hàm thực thi trong DAO Implement

```
@Override
public void insert(User user) {
String sql = "INSERT INTO [User](email, username, fullname, password, avatar, roleid,phone,
createddate) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
try {
    conn = new DBConnection().getConnection();
    ps = conn.prepareStatement(sql);
    ps.setString(1, user.getEmail());
    ps.setString(2, user.getUserName());
    ps.setString(3, user.getFullName());
    ps.setString(4, user.getPassWord());
    ps.setString(5, user.getAvatar());
    ps.setInt(6, user.getRoleid());
    ps.setString(7, user.getPhone());
    ps.setDate(8, user.getCreatedDate());
    ps.executeUpdate();
} catch (Exception e) {e.printStackTrace();}
}
```

## □ Bước 6: Viết hàm thực thi trong DAO Implement

```
@Override
public boolean checkExistEmail(String email) {
    boolean duplicate = false;
    String query = "select * from [user] where email = ?";
    try {
        conn = new DBConnection().getConnection();
        ps = conn.prepareStatement(query);
        ps.setString(1, email);
        rs = ps.executeQuery();
        if (rs.next()) {
            duplicate = true;
        }
        ps.close();
        conn.close();
    } catch (Exception ex) {}
    return duplicate;
}
```

## □ Bước 6: Viết hàm thực thi trong DAO Implement

```
@Override
public boolean checkExistUsername(String username) {
    boolean duplicate = false;
    String query = "select * from [User] where username = ?";

    try {
        conn = new DBConnection().getConnection();
        ps = conn.prepareStatement(query);
        ps.setString(1, username);
        rs = ps.executeQuery();
        if (rs.next()) {
            duplicate = true;
        }
        ps.close();
        conn.close();
    } catch (Exception ex) {}
    return duplicate;
}
```

## □ Bước 7: Viết Controller : RegisterController.java

```
@SuppressWarnings("serial")
@WebServlet(urlPatterns = "/register")
public class RegisterController extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        if (session != null && session.getAttribute("username") != null) {
            resp.sendRedirect(req.getContextPath() + "/admin");
            return;
        }
    }
}
```

### □ Bước 7: Viết Controller : RegisterController.java

```
// Check cookie
Cookie[] cookies = req.getCookies();
if (cookies != null) {
    for (Cookie cookie : cookies) {
        if (cookie.getName().equals("username")) {
            session = req.getSession(true);
            session.setAttribute("username", cookie.getValue());
            resp.sendRedirect(req.getContextPath() + "/admin");
            return;
        }
    }
}
req.getRequestDispatcher(Constant.Path.REGISTER).forward(req, resp);
}
```

## □ Bước 7: Viết Controller : RegisterController.java

```
@SuppressWarnings("static-access")
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
throws ServletException, IOException {
    resp.setCharacterEncoding("UTF-8");
    req.setCharacterEncoding("UTF-8");
    String username = req.getParameter("username");
    String password = req.getParameter("password");
    String email = req.getParameter("email");
    String fullname = req.getParameter("fullname");
    String phone = req.getParameter("phone");

    UserService service = new UserServiceImpl();
    String alertMsg = "";
```

## □ Bước 7: Viết Controller : RegisterController.java

```
if (service.checkExistEmail(email)) {  
    alertMsg = "Email đã tồn tại!";  
    req.setAttribute("alert", alertMsg);  
    req.getRequestDispatcher(Constant.Path.REGISTER).forward(req, resp);  
    return;  
}  
if (service.checkExistUsername(username)) {  
    alertMsg = "Tài khoản đã tồn tại!";  
    req.setAttribute("alert", alertMsg);  
    req.getRequestDispatcher(Constant.Path.REGISTER).forward(req, resp);  
    return;  
}
```

## □ Bước 7: Viết Controller : RegisterController.java

```
boolean isSuccess = service.register(username, password, email, fullname, phone);
if (isSuccess) {
    //SendMail sm = new SendMail();
    //sm.sendMail(email, "Shopping.iotstar.vn", "Welcome to Shopping. Please Login to use
    service. Thanks !");
    req.setAttribute("alert", alertMsg);
    resp.sendRedirect(req.getContextPath() + "/login");
} else {
    alertMsg = "System error!";
    req.setAttribute("alert", alertMsg);
    req.getRequestDispatcher(Constant.Path.REGISTER).forward(req, resp);
}
}
}
```

```
public static final String REGISTER = "/views/register.jsp";
```

- Yêu cầu: Người học thực hiện 02 ví dụ trên và nộp video quá trình thực hiện cho giảng viên.
- Thời hạn: 2 ngày